

Dossier de spécification PFR 2025 – 2026

**Université Paul Sabatier – Faculté des sciences de
l'ingénieur**

Groupe 5 : Delhem Sebastien, Gerain Antonin, Haegel
Lucas, Nosel Adrien, Raharitsifa Ny Aina.

Client : Ferrané Isabelle



Sommaire

Introduction.....	3
I. Présentation du projet.....	4
I.1 – Objectifs du document.....	4
I.2 – Objectifs du projet.....	4
II. Fonctionnalités.....	6
II.1 – Conversions de requêtes (opérateur → système).....	6
II.2 – Simulation des déplacements.....	9
II.3 – Interface utilisateur.....	11
II.4 – Traitement d'images.....	14
III. Organisation.....	19
III.1 – Outils utilisés.....	19
III.2 – Planning prévisionnel.....	20
III.3 – Répartition des rôles et des responsabilités....	21
Notes.....	22

Introduction

Dans le cadre de la première année de formation en Systèmes Robotiques et Industriels (SRI) à l'UPSSITECH de Toulouse, nous participons au Projet Fil Rouge (PFR). Nos professeur(e)s référent(e)s : Isabelle Ferrané, Julien Piquier et Julien Vanderstraeten sont les clients qui nous suivent tout au long de l'année. Le PFR permet de mobiliser et de combiner les compétences techniques et organisationnelles de chaque membre de l'équipe pour mener ce projet à la réussite.

Le projet nous offre aussi une opportunité de développer nos aptitudes en gestion de projet, en répondant aux attentes et exigences d'un client réel. Notre équipe est composée de cinq membres : Delhem Sébastien, Gerain Antonin, Haegel Lucas, Nosel Adrien et Raharitsifa Ny Aina. Nous travaillons avec Isabelle Ferrané, notre cliente référente.

L'objectif de cette année est de concevoir et de développer une plateforme mobile équipée de capteurs, capable d'effectuer des tâches manuelles, automatiques et commandées par écrit et à l'oral via une interface utilisateur. Le robot évolue dans un environnement évolutif avec des obstacles de couleurs et de formes différents. Tout le déroulement du projet doit respecter les contraintes définies par le client.

Le dossier de spécifications a pour but de présenter en détail les contraintes à prendre en compte ainsi que les fonctionnalités attendues du système, tout en respectant les besoins exprimés par le client.

I. Présentation du projet

I.1 – Objectifs du document

Le dossier de spécifications a pour objectif de définir l'ensemble des exigences fonctionnelles, techniques et organisationnelles nécessaires à la réalisation de la première phase du Projet Fil Rouge 2025-2026 (PFR1), menée par notre groupe.

Ce document décrit en détail :

- Les fonctionnalités à implémenter,
- Les méthodes et outils utilisés pour répondre aux besoins du cahier des charges,
- Ainsi que les bases de planification et de répartition des tâches au sein de l'équipe.

Il constitue une référence pour les différentes phases de développement, de test et de validation, en vue de produire des modules logiciels intégrables à la plateforme mobile lors de la deuxième phase du projet (PFR2).

De plus, ce dossier comprend :

- Les spécifications techniques de chaque module (conversion des requêtes, analyse d'images, etc.),
- Les interfaces attendues entre les composants du système,
- Et les critères de validation associés à chaque fonctionnalité.

Ces spécifications nous permettent d'avoir un fil conducteur pour le projet en respectant les attendus du client et en ayant une vision global des contraintes techniques et des délais impartis.

I.2 – Objectifs du projet

Ce projet a pour objectif de créer une plateforme mobile robotisée capable de se repérer dans l'espace grâce à différents capteurs (infrarouge, ultrason, caméra, etc.) et de se déplacer en évitant les obstacles et les murs présents dans une pièce.

I. Présentation du projet

Le robot possède deux modes de fonctionnement :

- Mode automatique : déplacement autonome dans un environnement évolutif.
- Mode manuel : pilotage via une interface utilisateur permettant à l'opérateur de donner des directions au robot. L'interaction avec le robot peut se faire par écrit au clavier ou vocalement grâce à un micro.

Ce projet est divisé en deux parties :

- PFR1 : mise en place du projet et développement logiciel avec simulation.
- PFR2 : assemblage et intégration logicielle sur la plateforme mobile.

Ce document porte uniquement sur les tâches à accomplir dans le cadre du PFR1.

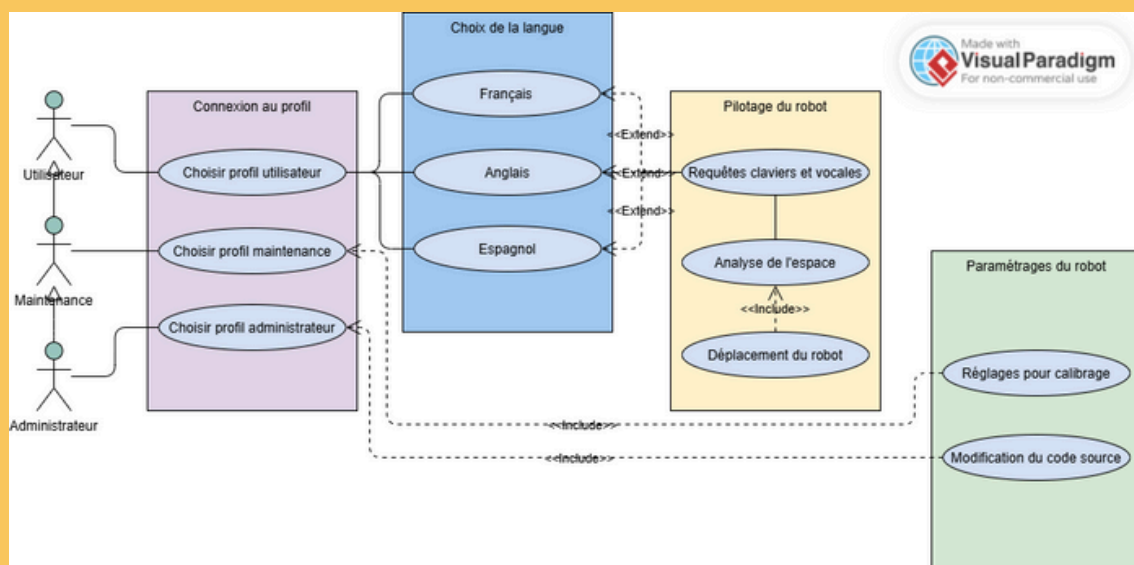


Figure 1 : Diagramme des cas d'utilisations du robot en mode manuel.

Le diagramme des cas d'utilisation (Use Case) a été réalisé sur le site Visual Paradigm Online. Ce site permet de créer différents types de diagrammes de manière intuitive, sans recours à l'intelligence artificielle.

Ce diagramme illustre le fonctionnement du pilotage du robot en mode manuel. Trois types d'utilisateurs peuvent interagir avec le robot, chacun disposant de niveaux de droits différents. Cette partie est détaillée dans la section **II.3 – Interface utilisateur**.

II. Fonctionnalités

II.1 – Conversions de requêtes (opérateur → système)

II.1.1 – Gestion des requêtes vocales

Dans le cadre du Projet Fil Rouge (PFR1), le développement de la gestion des requêtes vocales vise à assurer la transcription, la conversion et le traitement des commandes vocales afin de les rendre exploitables par le robot.

Les commandes vocales sont d'abord enregistrées et transcrites en texte grâce à un module de reconnaissance vocale fourni par le client.

Cette partie, dédiée à la conversion de la voix en texte, est déjà prise en charge par le module externe et ne fait donc pas partie des tâches à accomplir durant cette phase.

II.1.2 – Conversion requêtes texte en requêtes- commandes

Une fois la commande transcrite sous forme de texte dans l'interface homme-machine (IHM), la fonction de conversion doit transformer cette phrase en une instruction structurée compréhensible par le robot.

Cette étape comprend plusieurs sous-processus :

- Analyse linguistique et syntaxique :

Le programme doit analyser la phrase pour en extraire les éléments significatifs, identifier les groupes de mots utiles (définis dans un fichier de configuration syntaxique) et supprimer les mots superflus. Le résultat est un texte simplifié et structuré.

- Génération des commandes :

À partir du texte analysé, le système produit des commandes formatées selon le langage du robot, en associant chaque mot-clé à l'action correspondante.

II. Fonctionnalités

Exemples de conversion :

- "avance" → `forward(dist = default_distance)`
- "avance de 2 mètres" → `forward(dist = 2)`
- "trouve la balle rouge" → `where(object = ball, color = red)`
- Gestion des erreurs :
- Si une requête n'est pas comprise, le programme doit :
 - afficher un message d'erreur explicite sur le terminal (ex. mot inconnu, faute d'orthographe, mot hors langue choisie, syntaxe incohérente, etc.)

II.1.3 – Gestion des commandes

Afin d'exécuter les instructions successivement, le programme doit :

- Analyser la phrase pour identifier les différentes actions
- Exemple : "avance de 2 mètres et tourne à gauche" → deux commandes distinctes
- Reconnaître les actions et leurs paramètres associés
- Exemple : "avance de 2 mètres" → action = avancer, paramètre = 2 mètres
- Exécuter les actions dans l'ordre logique, en vérifiant la validité de chaque commande avant d'enchaîner sur la suivante

Le système doit aussi être capable de détecter les erreurs de syntaxe, les incohérences de commande et les paramètres manquants.

- Bibliothèque de mots-clés :

Les mots-clés essentiels incluent :

- "avance", "recule", "tourne", "droite", "gauche", "mètre", "mètres", "avant", "arrière", "trouve", "localise", "balle", "cube", "rouge", "vert", etc.

Une bibliothèque de synonymes sera ajoutée afin d'améliorer la flexibilité du système.

Les chiffres seront traités uniquement sous forme numérique (1, 2, 3, ...), et les unités seront abrégées ("m" pour mètre).

Exemples :

- "avance de 1 m, puis tourne à gauche et avance de 3 m"
- Commandes conditionnelles telles que :
 - "s'il y a une balle bleue, alors fais demi tours" seront considérées comme avancées et intégrées ultérieurement

II. Fonctionnalités

Commandes prises en charge par défaut :

- Commandes de base :
 - Avance → se déplacer vers l'avant.
 - Recule → se déplacer vers l'arrière.
 - Tourne à droite / à gauche → changer de direction.
 - Fais demi-tour → rotation complète.
 - Arrête → immobilisation totale.
- Commandes de déplacement précis :
 - "avance de deux mètres"
 - "avance jusqu'à la balle rouge"
 - "contourne l'obstacle par la droite", etc.
- Commandes avancées :
 - Accélère / Ralentis → ajustement de la vitesse.
 - Change de direction → nouvelle orientation.
 - Retourne au point de départ / Pause / Reprends.
- Commandes complexes (optionnelles) :
 - "contourne l'obstacle sans passer près de la balle rouge"
 - "passe à côté de la balle rouge puis de la balle bleue"
 - "cherche une balle rouge ou bleue"
 - "ne te positionne pas près de la balle verte"
 - "combien vois-tu de balles ?"

II.1.4 – Scénario de test / erreur

Pour valider la fiabilité du système, une simulation basée sur des commandes Unix est mise en place.

Elle permet de tester le comportement attendu des commandes avant leur intégration finale.

Les tests couvrent différents scénarios :

- Commandes simples
 - Exemple : avance, recule, tourne
 - Problèmes possibles : fautes d'orthographe, synonymes non reconnus, absence de feedback clair.

II. Fonctionnalités

- Commandes conditionnelles
 - Exemple : avance jusqu'à balle_rouge, contourne_obstacle droite
 - Problèmes possibles : objet non détecté, gestion incomplète des conditions, ambiguïté directionnelle
 - Enchaînements de commandes
 - Exemple : avance 2 ; tourne_droite 90 ; recule
 - Problèmes possibles : ordre d'exécution incorrect, arrêt inattendu, perte de l'état du robot
 - Gestion des erreurs et feedback utilisateur
 - Tout échec doit générer un message clair (ex. : Erreur : paramètre distance manquant)
 - Les erreurs doivent être journalisées (logs) afin de faciliter le débogage et l'amélioration continue
-

II.2 – Simulation des déplacements

Ce module présente en détail toutes les fonctionnalités prévues pour la simulation des déplacements, c'est-à-dire la gestion de l'activation des déplacements du système par l'utilisateur ainsi que la gestion des cas d'erreurs prévues par le système.

II.2.1 – Déplacement par commande utilisateur

L'utilisateur doit pouvoir gérer le déplacement du robot de deux façons différentes : commande textuelle et commande vocale.

- Chaque commande donnée par l'utilisateur devra être analysée et interprétée par le système pour effectuer l'action adéquate.
- La requête textuelle sera directement traitée par le système après que l'utilisateur ait validé sa saisie.
- La requête vocale sera dans un premier temps retransmise en texte pour ensuite être traitée par le système.

II. Fonctionnalités

II.2.2 Déplacement automatique

Le système doit pouvoir se déplacer automatiquement dans un environnement, avec pour objectif de suivre les bords de celui-ci.

- Une carte de l'environnement pourra être construite dynamiquement pendant son déplacement.
 - Pour lancer ou arrêter le déplacement automatique, l'utilisateur pourra utiliser la commande textuelle ou vocale.
-

II.2.3 Gestion des erreurs

Avant tout déplacement, le système devra analyser si celui-ci est possible et envoyer un retour à l'utilisateur en cas d'impossibilité.

- Un fichier log devra enregistrer et répertorier toutes les erreurs survenues.

Les erreurs de déplacement seront segmentées en deux parties distinctes :

1. Requête utilisateur impossible à interpréter :
 - La commande entrée par l'utilisateur n'est pas reconnue
 - La commande ne correspond pas à l'utilisation prévue du système ou n'a pas de sens
 2. Objectif de la requête impossible à satisfaire :
 - La requête a bien été interprétée, mais le système ne peut pas l'exécuter (objet non trouvable ou obstacle sur le chemin)
- Un retour du type d'erreur devra être fourni à l'utilisateur, qui pourra relancer sa requête sans relancer le système.

II. Fonctionnalités

II.3 – Interface utilisateur

L'interface homme machine (IHM) du robot permet de piloter et interagir avec le système grâce à des commandes textuelles et vocales.

Dans le cadre du PFRI, l'interaction se fait principalement via des menus textuels, des saisies au clavier et des commandes vocales.

II.3.1 – Gestion des langues

L'IHM du robot est disponible en plusieurs langues.

- À l'allumage, l'utilisateur doit sélectionner sa langue de préférence.

Langues	Niveaux de développement
Français, Anglais	1
Espagnol	2

Une fois la langue sélectionnée, l'ensemble du système adapte son interface, et toutes les requêtes textuelles et vocales doivent être formulées dans la langue choisie.

La maintenance et l'administrateur peuvent modifier le lexique et les paramètres de conversion pour assurer la cohérence avec de nouvelles configurations ou langues.

II.3.2 – Gestion des utilisateurs

Le système dispose d'une gestion des utilisateurs.

Cette gestion permet de créer différents profils avec des niveaux de droit différents.

II. Fonctionnalités

Profils	Droits
Opérateur	Formuler des requêtes, choisir les modes d'utilisation.
Maintenance (Niv.2)	Accès aux paramètres avancés, réparation et maintenance.
Administrateur	Modifier le code source et introduire des évolutions.

Une fenêtre permet de choisir son profil utilisateur et de saisir le mot de passe associé.

II.3.3 – Requêtes des déplacements

Le robot peut être piloté par requêtes textuelles ou requêtes vocales. Pour cela l'interface utilisateur possède un champ textuelle pour écrire les requêtes voulues par l'opérateur et un bouton pour activer le micro. Lorsque l'utilisateur utilise la commande vocale, la requête est automatiquement convertie en texte pour que l'opérateur puisse contrôler la conformité de ses propos. Seules les commandes configurées au préalable sont comprises par le système.

Pour voir les commandes disponibles, se référer à la partie **II.1 – Conversions de requêtes (opérateur → système)**.

II.3.4 – Configuration et connexion

Le système offre des options de paramétrage permettant à l'utilisateur de configurer :

- le traitement d'images (bits de quantification, plages de couleurs, détection des objets),
- la langue et le lexique pour la conversion des requêtes textuelles en commandes,
- la génération d'un fichier de log contenant l'historique des commandes exécutées, succès et échecs inclus.

Ces configurations permettent aux fonctionnalités développées dans le PFR1 de rester flexibles pour d'autres scénarios ou situations futures.

II. Fonctionnalités

II.3.5 – Scénarios de tests et gestion d’erreurs

Chaque requête et action utilisateur est validée avant exécution. Lorsqu’une situation est impossible à réaliser ou qu’une erreur survient lors de l’utilisation du robot, des messages d’erreurs (textuelles ou vocales) sont transmis à l’opérateurs.

Voici une liste de la gestion d’erreurs du robot et des scénarios possibles :

- Erreurs lors de l’authentification :
 - Si le nom d’utilisateur est incorrect : “Nom d’utilisateur inconnu. Veuillez réessayer.”
 - Si le mot de passe est incorrect : message d’erreur et nombre limité de tentatives (3 essais). Après échec répété, l’accès est temporairement bloqué pour des raisons de sécurité. Une clé de sécurité doit être saisie pour débloquer le système.
- Erreurs de saisie des requêtes :

Le système vérifie que chaque commande textuelle ou vocale est :

 - Syntaxiquement correcte : correspond aux commandes prévues.
 - Exécutable : réalisation des requêtes dans la configuration actuelle (absence d’obstacles, objets détectés, etc.)
- En cas d’erreur :
 - Le système informe l’utilisateur via synthèse vocale ou message texte :
 - “Commande inconnue”, “Déplacement impossible”, “Objet introuvable”
 - Si possible, le système propose des alternatives ou demande une clarification :
 - “Commande non reconnue. Veuillez réessayer.”
- Chaque erreur est enregistrée dans le fichier de log pour la traçabilité et les tests futurs.

Cette liste garantit que le robot réagit de manière prévisible aux erreurs, que l’utilisateur est informé et guidé, et que toutes les anomalies sont renseignées dans un fichier pour un suivi et débogage futur.

Lors des essais du robot, cette liste nous permettra d’établir un suivi des tests à réaliser pour permettre une utilisation fiable par la suite du projet.

II. Fonctionnalités

II.4 – Traitement d'images

II.4.1 – Objectif global

Créer un module C (ou C + Python) capable de :

- Charger une image couleur (format simple : PPM, BMP ou PNG converti).
 - Extraire les informations de couleur.
 - Détecter les zones homogènes (objets).
 - Caractériser chaque objet (forme, couleur dominante, position, taille).
 - Retourner les coordonnées du centre ou de la boîte englobante.
-

II.4.2 – Architecture modulaire proposée

A. Proposition d'organisation générale du code en modules C

- image.c – lecture et structure interne
- detection.c – algorithme de détection des zones homogènes
- analyse.c – caractérisation (couleur, forme, taille)
- utils.c – fonctions d'aide (calculs, conversions)

B. Langage et interfaçage

- Langage : C (impératif, structuré), Python (simulation PFRI)
 - Interface Python : via un script pour afficher les résultats (turtle)
-

II.4.3 – Étapes de traitement d'image

Étape 1 – Chargement et représentation interne

- Lire une image simple (PPM recommandé car facile à parser sans bibliothèque externe).
- Le format PPM est un format texte très basique contenant uniquement les valeurs des pixels (rouge, vert, bleu).
- On peut le lire facilement avec fscanf, sans avoir besoin d'une bibliothèque comme OpenCV.

II. Fonctionnalités

Représentation interne :

- R, G, B → trois tableaux 2D pour stocker séparément les valeurs rouge, verte et bleue de chaque pixel.
- Création de plusieurs fonctions pour :
 - lire le fichier et remplir la structure,
 - afficher ou enregistrer l'image,
 - appliquer des traitements (filtrage, conversion, etc.).

Étape 2 — Prétraitement

Proposition : conversion RGB → HSV (plus stable pour détecter les couleurs indépendamment de la luminosité).

En traitement d'image, chaque pixel est représenté par une couleur codée selon un modèle :

- RGB (Rouge, Vert, Bleu) → modèle "écran", brut, lié à l'intensité lumineuse des couleurs primaires.
 - En RGB, une même couleur vue sous des conditions de lumière différentes aura des valeurs très variables.
 - Exemple : un rouge éclairé et un rouge dans l'ombre auront des triplets (R,G,B) très différents.
- HSV (Teinte, Saturation, Valeur) → modèle "perceptif", proche de la perception humaine.
 - Hue (H) : la teinte (rouge, vert, bleu, jaune...), exprimée en degrés (0° à 360°).
 - Saturation (S) : la "pureté" de la couleur (de 0 à 1).
 - Value (V) : la luminosité (de 0 à 1).

En HSV, la teinte (H) reste constante même si la luminosité change.

Cela permet une détection de couleur plus robuste aux variations d'éclairage.

Étape 3 — Solution de filtrage

Objectif : réduire le bruit tout en préservant les vraies structures.

Filtrage par moyenne (moyennage)

- On remplace chaque pixel par la moyenne de ses 9 voisins (fenêtre 3×3).
- Permet de lisser les variations rapides → réduction du bruit mais perte de détails.

II. Fonctionnalités

Exemple :

Valeurs :

120 122 118

123 255 119

121 120 122

Moyenne $\approx 135 \rightarrow$ le pixel central bruité (255) devient 135.

Filtrage par médiane

- On prend la valeur médiane parmi les 9 pixels.
- Plus robuste contre le bruit impulsionnel (pixels isolés très clairs ou sombres).

Exemple :

Valeurs triées : 118, 119, 120, 120, 121, 122, 122, 123, 255 $\rightarrow$ médiane = 121

$\rightarrow$ Le pixel bruité 255 devient 121 : nettoyage efficace sans flou excessif.

Étape 4 — Détection d'objets

4.1 – Méthode basique : seuillage par couleur

Définir pour chaque couleur recherchée une plage HSV dans un fichier de configuration :

red_min_H=0 red_max_H=10

red_min_S=0.6 red_max_S=1

red_min_V=0.3 red_max_V=1

4.2 – Segmentation et regroupement des pixels

- Parcourir l'image et créer un masque binaire pour chaque couleur cible.
- Utiliser un algorithme de flood fill / region growing pour regrouper les pixels voisins d'une même couleur.

Pour chaque région détectée :

- Nombre de pixels
- Coordonnées min/max $\rightarrow$ boîte englobante
- Coordonnées moyennes $\rightarrow$ centre

II. Fonctionnalités

Étape 5 — Caractérisation

Pour chaque objet détecté :

- Couleur dominante : moyenne des valeurs HSV des pixels.
- Forme : rapport largeur/hauteur ou moment d'inertie approximé.
 - ex. rapport $\approx 1 \rightarrow$ cercle
 - rapport $> 1.5 \rightarrow$ rectangle/ligne
- Taille relative : surface (nombre de pixels).
- Position : centre de gravité (x, y).

Étape 6 — Résultats et communication

- Générer une structure de résultat.
- Écrire les données dans un fichier .txt ou .json.
- (Facultatif) Créer un script Python [visualisation.py](#) pour surligner les objets détectés (via matplotlib ou PIL).

II.4.4 – Modules / fonctions à écrire soi-même

Module	Contenu	Pourquoi
image.c	Lecture / écriture d'images PPM	Manipulation bas niveau
color.c	Conversion RGB $\leftrightarrow$ HSV, comparaison de teintes	Comprendre la couleur
segmentation.c	Flood fill, étiquetage de composantes	Algorithmique impérative
analyse.c	Calcul centre, forme, taille	Traitement numérique
config.c	Lecture fichiers de configuration (.ini / .txt)	Modularité
resultats.c	Écriture des résultats	Intégration (PFR2) future

II. Fonctionnalités

II.4.5 – Bibliothèques autorisées et utiles

- Objectif : valider nos compétences en C, donc pas d'OpenCV
 - Bibliothèques possibles :
 - `stdlib.h`, `stdio.h`, `math.h`, `string.h`
 - `dirent.h` pour lire automatiquement toutes les images d'un dossier
 - `json-c` (optionnel) pour format structuré
 - `python.h` si interaction avec Python pour l'affichage
-

II.4.6 – Extensions possibles (bonus)

- Détection de plusieurs objets de même couleur
- Association couleur-forme (ex. "balle rouge" → cercle + rouge)
- Création d'un jeu de tests automatiques : images → résultats attendus
- Mode simulation : afficher le déplacement du robot vers l'objet détecté

III. Organisation

III.1 – Outils utilisés

Afin d'assurer le bon déroulement du Projet Fil Rouge, différents outils logiciels ont été sélectionnés pour couvrir l'ensemble des besoins en développement, gestion de version et planification.

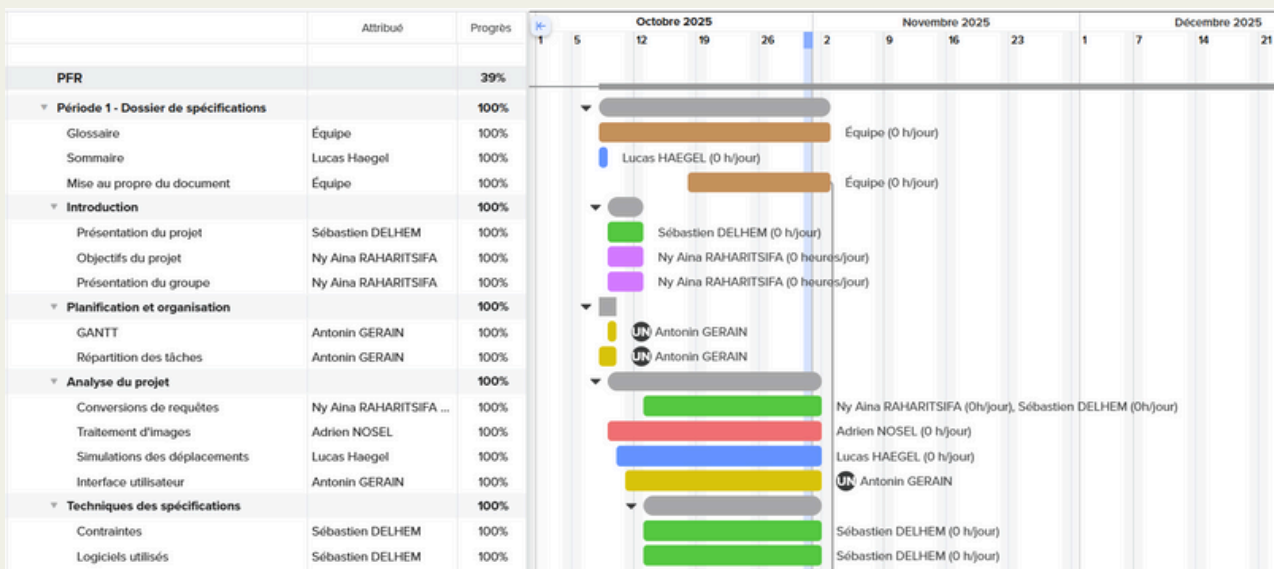
Catégorie d'outil	Nom	Détails / Version
Systèmes d'exploitation	Windows 11	Utilisé pour le développement et les tests sur environnement graphique.
	Debian (UNIX)	Utilisé pour les tests en environnement Linux et la compatibilité multiplateforme.
	macOS Sequoia	Utilisé pour la vérification de la portabilité du projet.
Environnements de développement	Visual Studio Code	Version 1.105.0 – Utilisé pour le développement principal. • Python 3.13.6 • C C23 • Jupyter 7.4.7
Gestionnaires de configuration Web	GitHub	Version 3.6 – Plateforme d'hébergement du code et suivi collaboratif.
	Git	Version 2.51.0 – Gestion locale des versions et synchronisation avec GitHub.

III. Organisation

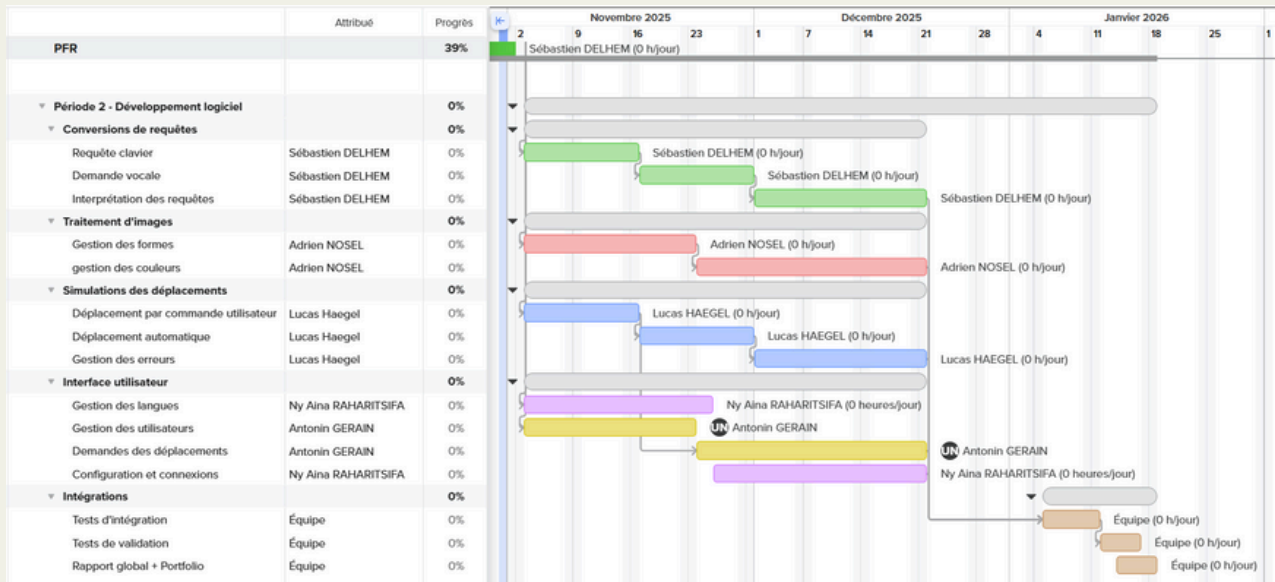
Outils de gestion de projet	Trello	Utilisé pour la planification des tâches, le suivi d'avancement et la répartition du travail au sein de l'équipe.
Virtualisation Simulation	VirtualBox	Version 7.1.12 – Environnement de travail utilisant Debian.

III.2 – Planning prévisionnel

Pour planifier les tâches du projet, nous avons utilisé le site internet TeamGantt. Ce site permet de réaliser des diagrammes de GANTT personnalisables sans utilisation de l'intelligence artificielle. Le site permet d'affecter des tâches aux différents membres du groupe et de compléter le pourcentage de réalisation des ces dernières. Le logiciel a une fonctionnalité de calcul de temps de travail, il ne faut pas le prendre en compte ici.



III. Organisation



III.3 – Répartition des rôles et des responsabilités

Nom et prénom	Rôle dans le projet	Responsabilité des tâches
DELHEM Sébastien	Chargé de communication, développeur	Conversion de requête
GERAIN Antonin	Chef de projet, développeur	Interface utilisateur
HAEGEL Lucas	Développeur	Simulation des déplacements
NOSEL Adrien	2eme chargé de communication (remplacement), développeur	Traitement d'image
RAHARITSIFA Ny Aina	Développeur	Interface utilisateur

Notes

Pour la réalisation de ce document, une intelligence artificielle générative a été utilisée pour diverses fonctions, citées ci-après :

- Correction des fautes d'orthographe et de grammaire ;
- Harmonisation de certains éléments de texte afin de rendre le format du document plus uniforme ;
- Organisation de certaines idées pour les rendre plus compréhensibles pour le lecteur.

L'IAG principale utilisée est ChatGPT. À chaque utilisation de l'IA, le contenu généré a été relu, corrigé et modifié afin d'éviter toute dérive et de conserver un contrôle total sur la rédaction. L'intelligence artificielle a été utilisée comme un outil de soutien à la rédaction de ce rapport, et non comme un substitut à notre travail.

Notre groupe vous remercie pour votre confiance quant à l'usage raisonné et transparent que nous avons fait de l'IAG dans le cadre de la rédaction du document de spécification.

